计算机系统概论 lab1 实验报告

25800190042 赵婉扬

1. signMask

   将 0000 0000……0001 向左平移 31 位，右侧自动补 0，则变成 1000 0000……0000

2. bitXor

   原本 x ^ y = (x & ~y) | (~x & y)，由于不能用 | ，因此利用 A | B = ~(~A & ~B)，代入 A = x & ~y，B = ~x & y，得到~(~(x & ~y) & ~(~x & y))

3. negativePart

   先令 mask=x<<31，使得所有正数的 mask 全 0，所有负数的 mask 全 1。然后再用(~x+1)求负数的相反数，&上 mask 做过滤即可。

```
int negativePart(int x){
  int mask = x >> 31;
  return mask & (~x + 1);
}
```

4. copyByteWithin

   思路是，先用 byte 取出目标字节，然后再挪动 byte 到目标位置（其余位置为 0）作为 shifted，然后再用掩码 mask 把原数字的目标位置的字节变成全 0，再与 shifted 做或运算即可。

```
int copyByteWithin(int x, int src, int dst) {
  int byte = (x >> (src << 3)) & 0xFF;
  int shifted = byte << (dst << 3);
  int mask = ~(0xFF << (dst << 3));
  return (x & mask) | shifted;
}
```

5. logicalShift

   先把 x 算术右移 n 位，然后造一个头 n 位全 0 的掩码，做&即可。

```
int logicalShift(int x, int n) {
  int shifted = x >> n;
  int mask = ~(((1 << 31) >> n) << 1);
  return shifted & mask;
}
```

   不过这里要注意 mask 不能直接<<n-1，否则 n=0 时违规。

6. swapNibblePairs

   先用 0000……00001111 左移不同位数再拼起来得到一个掩码 low，用 x&low 再左移 4 位相当于把低 4 位放到高 4 位的位置，然后再把 x 右移 4 位再与 low 做&相当于把高 4 位放到低 4 位的位置，做 | 即可。

```
int swapNibblePairs(int x) {
  int low = 0x0F | (0x0F << 8) | (0x0F << 16) | (0x0F << 24);
  return ((x & low) << 4) | ((x >> 4) & low);
}
```

7. secondLowestZeroBit

   首先依据经典写法，最低的 0 位 = ~x & (x + 1)。那么我们的思路就是先把最低 0 位填成 1，然后再找一遍最低 0 位，就能找到原本第二低的 0 位了。

```
int secondLowestZeroBit(int x) {
  int t = x | (x + 1);
  return ~t & (t + 1);
}
```

8. oddParity

   容易证明，一些位做异或，最终得到的结果由这些位中 1 的个数的奇偶性决定。因此我们只需要设计一种算法能够使 32 位全部做异或然后按照要求决定是否取反就可以了。思路是，把 x 右移 16 位再与原本的 x 做异或，相当于把高 16 位与低 16 位一一做异或然后存储在低 16 位中，此时高 16 位不管；以此类推，右移 8，4，2，1，得到的最低位就可以表征 1 的个数奇偶性。

```
int oddParity(int x) {
  x = x ^ (x >> 16);
  x = x ^ (x >> 8);
  x = x ^ (x >> 4);
  x = x ^ (x >> 2);
  x = x ^ (x >> 1);
  // 偶数个 1 返回 1，奇数个 1 返回 0
  return ~x & 1;
}
```

9. rotateRightBits

   思路是，先把 x 逻辑右移 n%32 位，然后再把原 x 左移 32-n%32 位，两者做或运算即可。

```
int rotateRightBits(int x, int n) {
  int m = n & 31;
  int s = (32 + ~m + 1) & 31;
  int mask = ~((~0) << s);
  return ((x >> m) & mask) | (x << s);
}
```

   其中，&31 相当于%32 的作用。不直接用 32-n%32 是因为，32-n%32 有可能等于 32，会导致<<s 是未定义行为。

10. roundEvenPow2

    思路是，x = base * 2^n + rem， 0 <= rem < 2^n，然后根据 rem 是否足够 2^(n-1)以及 base 的奇偶来判断取 base 还是 base+1。

```
int roundEvenPow2(int x, int n) {
  int mask = (1 << n) + ~0; // 构造低 n 位全 1 的掩码，用于取余数
  int half = 1 << (n + ~0); // 计算半程值 2^(n-1)
  int rem = x & mask; // 取出 x 除以 2^n 的余数
  int base = x >> n; // 取出 x 除以 2^n 的商（向下取整）
  int geHalf = ((rem + half) >> n) & 1; // 判断余数是否大于等于半程：是则 1，否则 0
  int tie = !(rem ^ half); // 判断是否恰好在中点：是则 1，否则 0
  int odd = base & 1; // 判断商是否为奇数：奇数则 1，偶数则 0
  int tieMask = ~tie + 1; // 中点时生成全 1 掩码，否则全 0
  int up = (tieMask & odd) | (~tieMask & geHalf);
  // 决定是否进位：中点看商奇偶，非中点看余数是否过半
  return (base + up) << n; // 进位后的商乘以 2^n，得到最终结果
}
```

11. midpointTowardFirst

思路是，设 mid=（x+y）/2，则当且仅当 x+y 为奇数且 x>y 时才需要取 mid+1。

```
int midpointTowardFirst(int x, int y) {
  int mid = (x & y) + ((x ^ y) >> 1); // 计算 floor((x+y)/2)，不会溢出
  int odd = (x ^ y) & 1;              // 判断 x+y 是否为奇数：奇数则 odd=1
  int sx = x >> 31;                   // 取 x 的符号位：非负为 0，负为 -1（全1）
  int sy = y >> 31;                   // 取 y 的符号位
  int diff = x + ~y + 1;              // 计算 x - y（~y+1 就是 -y）
  int same = ~(sx ^ sy);              // 判断 x、y 是否同号：同号则全1，异号则全0
  int gt = (same & ~(diff >> 31)) | (~same & ~sx); // 计算 x > y 的条件，gt 全1表示 x>y
  return mid + (odd & gt & 1);
  // 若 x+y 为奇数且 x>y，则中点向上取整（mid+1），否则返回 mid
}
```

12. isBetweenEitherOrder

思路是，x 在 a，b 闭区间之内等价于 x-a 与 x-b 异号或 x=a 或 x=b。

```
int isBetweenEitherOrder(int x, int a, int b) {
  int ax = a + ~x + 1; // 计算 a - x
  int bx = b + ~x + 1; // 计算 b - x
  int xor_ax = a ^ x; // a 和 x 的异或，用于判断是否异号
  int xor_bx = b ^ x; // b 和 x 的异或
  int sign_ax = ((xor_ax >> 31) & (a >> 31)) | (~(xor_ax >> 31) & (ax >> 31));
  // 修正 ax 的真实符号位
  int sign_bx = ((xor_bx >> 31) & (b >> 31)) | (~(xor_bx >> 31) & (bx >> 31));
  // 修正 bx 的真实符号位
  int diff = (sign_ax ^ sign_bx) >> 31; // 如果两个符号位不同，diff 为全 1，否则为 0
  int zero = !ax | !bx; // 如果 ax == 0 或 bx == 0，说明 x 等于 a 或 b
  return (diff & 1) | zero; // 符号不同或等于端点时返回 1，否则返回 0
}
```

这里需要注意的是，当 x-a 和 x-b 溢出的时候，符号会翻转，所以我们需要用 sign_ax 和 sign_bx 来额外判断一下。

13. mul5Sat

计算 x*5 = (x<<2)+x；用阈值加法检测溢出：非负且 x > 0x19999999 时正溢出，负且 x < 0xE6666667 时负溢出；得到全 1/全 0 掩码后，溢出时按方向返回 INT_MAX 或 INT_MIN，未溢出返回原结果。

```
int mul5Sat(int x) {
  int posLimit = 0x19 | (0x99 << 8) | (0x99 << 16) | (0x19 << 24);
  // 正溢出阈值 0x19999999
  int negLimit = ~posLimit;
  // 负溢出阈值相关值 0xE6666666
  int sx = x >> 31; // x 的符号位：0 表示非负，-1 表示负
  int posOvf = ~sx & ~((x + negLimit) >> 31); // 正溢出：x 非负且 x > 0x19999999
  int negOvf = sx & ((x + posLimit) >> 31); // 负溢出：x 为负且 x < 0xE6666667
  int ovf = posOvf | negOvf; // 溢出标志：全 1 表示溢出，全 0 表示未溢出
  int intMax = ~(1 << 31); // INT_MAX = 0x7FFFFFFF
  int intMin = 1 << 31; // INT_MIN = 0x80000000
  int sat = (posOvf & intMax) | (negOvf & intMin); // 根据溢出方向选择饱和值
  int res = (x << 2) + x; // 计算 x*5 的 32 位结果
  return (ovf & sat) | (~ovf & res); // 溢出则返回饱和值，否则返回计算结果
}
```

14. classifyAdd3

思路是，用无符号进位模拟 64 位加法，算出高 32 位 hi 和低 32 位 s2，根据 hi 的值

和 s2 的符号组合判断正溢出、负溢出或无溢出，返回 1、-1 或 0。

```
int classifyAdd3(int x, int y, int z) {
  int s1 = x + y;                                    // x+y 的低 32 位
  int c1 = (((x & y) | ((x | y) & ~s1)) >> 31) & 1;   // x+y 的无符号进位（0 或 1）
  int s2 = s1 + z;                                   // (x+y)+z 的低 32 位
  int c2 = (((s1 & z) | ((s1 | z) & ~s2)) >> 31) & 1;// 第二次加法的无符号进位（0 或 1）
  int sx = x >> 31;                                  // x 的符号扩展（0 或 -1）
  int sy = y >> 31;                                  // y 的符号扩展
  int sz = z >> 31;                                  // z 的符号扩展
  int hi = sx + sy + sz + c1 + c2;                   // 64 位和的高 32 位
  int hip1 = hi + 1;                      // hi + 1，方便判断 hi == -1 和 hi < -1
  int hiPos = ~(hi >> 31) & !!hi;                    // hi > 0 时为 1
  int hiZero = !hi;                                  // hi == 0 时为 1
  int hiNegOne = !hip1;                              // hi == -1 时为 1
  int hiLtNegOne = (hip1 >> 31) & !!hip1;            // hi < -1 时为 1
  int s2neg = (s2 >> 31) & 1;                        // s2 < 0 时为 1
  int posOvf = hiPos | (hiZero & s2neg);             // 正溢出：hi>0，或 hi==0 且 s2<0
  int negOvf = hiLtNegOne | (hiNegOne & !s2neg);  // 负溢出：hi<-1，或 hi==-1 且 s2>=0
  return posOvf | (negOvf << 31 >> 31);     // 正溢出返回 1，负溢出返回 -1，无溢出返回 0
}
```

这个题的难点在于：进位公式算出 x+y 和 (x+y)+z 的进位后，>>31 是算术右移，最高位为 1 时得到的是 -1 而不是 1，必须再 &1 规范成 0 或 1，否则 hi 会算错；同时 !!hi 这种写法结果只是 0 或 1，不是全 0 或全 1 的掩码，拿它去 & 别的值只会保留最低位，导致后续判断全错，需要返回 -1 时应当用 negOvf << 31 >> 31 生成全 1；此外判断条件不能只看 hi != 0，因为 hi == -1 且低 32 位 s2 >= 0 时是负溢出，但 hi == -1 且 s2 < 0 时可能没溢出，而 hi == 0 且 s2 < 0 时反而是正溢出，所以必须把 hi 的值和 s2 的符号组合起来判断；最后不能直接看 x+y+z 的符号，因为中间结果可能已经溢出，s2 的符号不代表真实和的符号，必须模拟 64 位加法，用高 32 位 hi 和低 32 位 s2 一起判断。

15. floatScaleThreeHalves

思路是，把输入拆成符号、阶码、尾数三部分，先判断 NaN/Inf/±0 直接返回；否则把非规格化数和规格化数统一还原成尾数 mant 和实际指数 e，然后计算 mant * 3，再除以 2 并对结果做 round-to-nearest-even 舍入；舍入后如果尾数超过 24 位就右移一位并把指数加一，最后根据新的尾数是否达到 1 << 23 决定写成规格化数还是非规格化数，同时更新阶码并处理溢出为无穷。

```c
unsigned floatScaleThreeHalves(unsigned uf) {
  unsigned sign = uf & 0x80000000; // 取出符号位
  unsigned exp = (uf >> 23) & 0xFF; // 取出 8 位阶码
  unsigned frac = uf & 0x7FFFFF; // 取出 23 位尾数

  if (exp == 0xFF) return uf; // NaN 或 Inf，直接返回原值
  if (exp == 0 && frac == 0) return uf; // ±0，直接返回原值

  unsigned mant; // 用来保存还原后的尾数
  int e; // 实际指数
  if (exp == 0) { // 非规格化数
    mant = frac; // 尾数就是 frac，没有隐藏的 1
    e = -126; // 非规格化数的实际指数固定为 -126
  } else { // 规格化数
    mant = (1 << 23) | frac; // 加上隐藏的最高位 1
    e = exp - 127; // 实际指数 = 阶码 - 偏移量 127
  }

  unsigned prod = mant * 3; // 先计算 mant * 3
  unsigned q = prod >> 1; // 除以 2 的整数部分
  unsigned r = prod & 1; // 除以 2 的余数（是否丢了 0.5）

  unsigned result; // 舍入后的尾数结果
  int exp_inc = 0; // 指数是否需要加 1

  if (q < (1u << 24)) { // 如果 q 还没超过 24 位
    result = q + (r & q & 1); // round-to-nearest-even：余数为 1 且 q 为奇数时进位
  } else { // q 超过 24 位，需要再右移一位
    unsigned q2 = q >> 1; // 再除以 2 的整数部分
    unsigned r2 = q & 1; // 再除以 2 的余数
    result = (r == 0) ? (q2 + (r2 & q2 & 1)) : (q2 + r2); // 根据 r 分情况舍入
    exp_inc = 1; // 因为右移了一位，指数要加 1
  }

  e += exp_inc; // 更新实际指数

  if (result >= (1u << 23)) { // 如果尾数达到 24 位，说明是规格化数
    exp = e + 127; // 重新计算阶码
    if (exp >= 0xFF) return sign | 0x7F800000; // 阶码上溢，返回无穷
    if (exp <= 0) return sign | result; // 阶码下溢，返回非规格化数形式
    return sign | (exp << 23) | (result & 0x7FFFFF); // 拼回规格化浮点数
  } else { // 否则是非规格化数
    return sign | result; // 直接拼回非规格化浮点数
  }
}
```

这个题，难点在于 3/2 不是简单的移位能表达的，必须先算出 mant * 3 再除以 2，中间还要做最近偶数舍入；而且舍入不能先把最低位丢掉再判断，否则像 0x3f7ffffa 这种正好卡在中间的值会少进一位，正确做法是先拿到 q = prod >> 1 和 r = prod & 1，再结

合 q 的奇偶决定是否进位；另外非规格化数和规格化数要统一处理，尾数进位后还要重新规格化并更新阶码，最后还要单独处理 exp == 0xFF 的 NaN/Inf 和阶码上溢的情况。

16. floatRoundEven

思路是，先拆出符号、阶码、尾数；NaN/Inf 直接返回；非规格化数和 |v|<0.5 舍入到带符号的 0；e==-1 时判断是否恰好 0.5，是则舍入到 0，否则舍入到 1；e>=0 时，把浮点值写成 (1 << e) + (frac >> shift) 加上小数部分，按最近偶数舍入到整数 base，再把 base 重新规格化成单精度浮点表示。

```
unsigned floatRoundEven(unsigned uf) {
  unsigned sign = uf & 0x80000000;
  unsigned exp = (uf >> 23) & 0xFF;
  unsigned frac = uf & 0x7FFFFF;

  if (exp == 0xFF) return uf; // NaN 或 Inf，原样返回
  if (exp == 0) return sign;  // 非规格化数，绝对值 < 2^-126，舍入到 0

  int e = exp - 127;
  if (e >= 23) return uf;     // 已经是整数，不需要舍入
  if (e < -1) return sign;    // |v| < 0.5，舍入到 0

  if (e == -1) {
    // v = 0.5 + frac/2^24
    if (frac == 0) return sign;         // 恰好 0.5，舍入到偶数 0
    return sign | 0x3F800000;           // 大于 0.5，舍入到 1.0
  }

  // e >= 0
  int shift = 23 - e;
  unsigned rem = frac & ((1 << shift) - 1);   // 小数部分
  unsigned half = 1 << (shift - 1);           // 半程
  unsigned base = (1 << e) + (frac >> shift); // 整数部分的值（含隐含的 1）
```

```
  // 最近偶数舍入
  if (rem > half || (rem == half && (base & 1))) {
    base++;
  }

  if (base == 0) return sign;

  // 将整数 base 重新转换为浮点数
  int k = 31;
  while (!(base & (1 << k))) k--; // 找最高位
  int E = k;
  unsigned new_frac;
  if (k <= 23) {
    new_frac = (base << (23 - k)) & 0x7FFFFF;
  } else {
    new_frac = (base >> (k - 23)) & 0x7FFFFF;
  }
  int new_exp = E + 127;
  if (new_exp >= 0xFF) return sign | 0x7F800000;
  return sign | (new_exp << 23) | new_frac;
}
```

需要注意的是，整数部分必须加上隐含的 1，不能只用尾数高位；舍入后 base 可能进位到 24 位，需要重新找最高位并计算新的阶码和尾数；e==-1 的 0.5 边界要单独处理；同时注意 e>=23 时原数已是整数，直接返回。

17. float_i2f

先特判 0 直接返回；取出符号位并对负数取绝对值（~x + 1 对 INT_MIN 也成立），然后从高到低找到最高有效位 e；若 e <= 23，把绝对值左移对齐到 23 位尾数即可；若 e > 23，先右移 e-23 位保留高位，同时用移出的低位做 round-to-nearest-even 舍入，若尾数进位到第 24 位，则尾数清零、指数加一；最后用 e + 127 算阶码，拼回符号、阶码、尾数，阶码上溢则返回无穷。

```c
unsigned float_i2f(int x) {
  if (x == 0) return 0;
  unsigned sign = (x >> 31) & 0x80000000; // 符号位
  unsigned abs = x;
  if (x < 0) abs = ~x + 1; // 取绝对值，INT_MIN 也正确
  int e = 31;
  while (!(abs & (1 << e))) e--; // 找最高有效位
  unsigned frac;
  if (e <= 23) {
    frac = (abs << (23 - e)) & 0x7FFFFF; // 左移对齐，无需舍入
  } else {
    int shift = e - 23;
    unsigned shifted = abs >> shift; // 保留的高位
    unsigned rem = abs & ((1 << shift) - 1); // 移出的低位
    unsigned half = 1 << (shift - 1); // 半程
    frac = shifted & 0x7FFFFF; // 取低 23 位作为尾数
    // 最近偶数舍入
    if (rem > half || (rem == half && (frac & 1))) {
      frac++;
      if (frac & 0x800000) { // 尾数进位到第 24 位
        frac = 0; // 尾数清零
        e++; // 指数加一
      }
    }
  }
  unsigned exp = e + 127;
  if (exp >= 0xFF) return sign | 0x7F800000; // 溢出为无穷
  return sign | (exp << 23) | frac;
}
```

难点如下：一是 INT_MIN 取绝对值不能简单用 -x，要用 ~x + 1 才能得到正确的 0x80000000；二是 e > 23 时不能直接截断，必须用移出的低位和半程比较实现最近偶数舍入，并且注意半程时还要看尾数最低位是否为奇数；三是舍入后尾数可能进位到 1 << 23，此时不能简单右移，而应把尾数清零、指数加一；四是阶码计算后要检查是否达到 0xFF，避免溢出为无穷时位表示错误。

18. bitCount

用分治并行加法统计 1 的个数：先用 m1 把相邻两位分成一组，组内相加得到每 2 位的 1 的个数；再用 m2、m4 依次把相邻的 2 位组、4 位组相加，得到每 4 位、每 8 位的计数；最后用 m8 把高 16 位和低 16 位相加，再用 & 0x3F 截取最终结果，因为 32 位数中 1 的个数最多 32，需要 6 位。

```
int bitCount(int x) {
  int m1 = 0x55 | (0x55 << 8) | (0x55 << 16) | (0x55 << 24);
  int m2 = 0x33 | (0x33 << 8) | (0x33 << 16) | (0x33 << 24);
  int m4 = 0x0F | (0x0F << 8) | (0x0F << 16) | (0x0F << 24);
  int m8 = 0xFF | (0xFF << 16);
  x = (x & m1) + ((x >> 1) & m1);
  x = (x & m2) + ((x >> 2) & m2);
  x = (x & m4) + ((x >> 4) & m4);
  x = (x + (x >> 8)) & m8;
  x = (x + (x >> 16)) & 0x3F;
  return x;
}
```

难点有这些。一是常量限制只能写 0–255，所以 0x55555555、0x33333333、0x0F0F0F0F 这些掩码必须用 0x55、0x33、0x0F 反复左移拼出来；二是每步相加后必须用对应掩码清掉不属于当前组的位，否则相邻组的计数会互相干扰；三是右移是算术右移，负数右移会补符号位，但因为后面紧跟着 & m1、& m2 等掩码，只保留有效位，所以不影响结果；四是操作数要控制在 40 以内，掩码构造和 5 步折叠都要数清楚。

19. bitReverse

用分治逐层交换：先交换相邻 1 位，再 2 位、4 位、8 位，最后交换高低 16 位。每步用掩码取出要移动的位，移位后合并。

```
int bitReverse(int x) {
  int m1 = 0x55 | (0x55 << 8) | (0x55 << 16) | (0x55 << 24); // 0x55555555
  int m2 = 0x33 | (0x33 << 8) | (0x33 << 16) | (0x33 << 24); // 0x33333333
  int m4 = 0x0F | (0x0F << 8) | (0x0F << 16) | (0x0F << 24); // 0x0F0F0F0F
  int m8 = 0xFF | (0xFF << 16);                              // 0x00FF00FF
  int m16 = 0xFF | (0xFF << 8);                              // 0x0000FFFF

  x = ((x >> 1) & m1) | ((x & m1) << 1); // 交换相邻 1 位
  x = ((x >> 2) & m2) | ((x & m2) << 2); // 交换相邻 2 位
  x = ((x >> 4) & m4) | ((x & m4) << 4); // 交换相邻 4 位
  x = ((x >> 8) & m8) | ((x & m8) << 8); // 交换相邻 8 位
  x = (x << 16) | ((x >> 16) & m16);     // 交换高低 16 位，逻辑右移
  return x;
}
```

这里最后一步必须用逻辑右移，否则负数会被符号扩展污染结果；m16 就是用来把 x >> 16 的高位清掉的掩码。